一、实验题目

利用pytorch框架搭建神经网络实现中药图片分类，其中中药图片数据分为训练集 train 和测试集 test ，训练集仅用于网络训练阶段，测试集仅用于模型的性能测试阶段。训练集和测试集均包含五种不同类型的中药图片： baihe、dangshen 、 gouqi 、 huaihua 、 jinyinhua 。请合理设计神经网络架构，利用训练集完成网络训练，统计网络模型的训练准确率和测试准确率，画出模型的训练过程的loss曲线、准确率曲线。

二、实验内容

1.算法原理

利用卷积神经网络CNN来实现中药图片分类。

一个卷积神经网络包括：输入层、卷积层、池化层、全连接层和输出层

1.1 输入层

输入图像信息

1.2 卷积层

利用卷积计算，提取图像特征

1.3 池化层

通常在连续的卷积层之间会周期性地插入一个池化层，通过降低数据的尺寸，减少计算资源，同时能够防止过拟合。

池化层有两种类型：最大池化和平均池化

1.4 全连接层

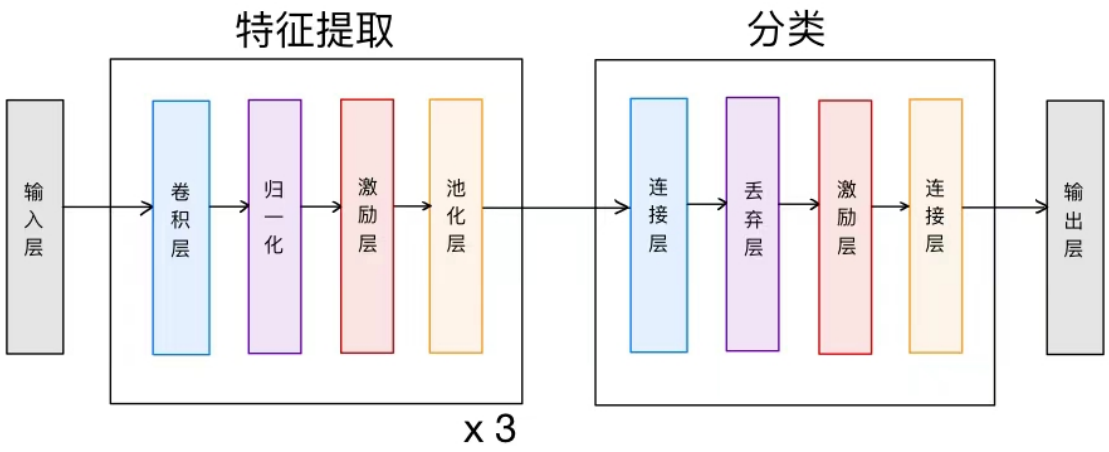
全连接层将最终得到的所有特征矩阵转化成一维的特征向量

1.5 输出层

将全连接层得到的一维向量经过计算后得到识别值的一个概率

最终，我的卷积神经网络结构设计如下：

（其中，x 3 表示类似的结构有3个）



1.6 损失函数

采用**交叉熵**，表达式如下：

$$H(p,q)=\sum_{i=1}^{n}p\left(x_i\right)\log\frac{1}{q\left(x_i\right)}=-\sum_{i=1}^{n}p\left(x_i\right)\log q\left(x_i\right)$$

利用交叉熵作为损失函数，适合于多分类问题。

2.关键代码展示

2.1 卷积神经网络结构

在我定义的卷积神经网络中，共有3层卷积层。在得到全连接层之前丢弃部分神经元，以防止过拟合。

```
out1_size=16
out2_size=32
out3_size=64

class CNN(nn.Module):
    def __init__(self,num_classes):
        super().__init__()
        self.conv1=nn.Sequential(

 nn.Conv2d(in_channels=3,out_channels=out1_size,kernel_size=3,stride=1,padding=1)
,
            nn.BatchNorm2d(out1_size),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(in_channels=out1_size, out_channels=out2_size,
kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(out2_size),
```

```
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.conv3=nn.Sequential(
            nn.Conv2d(in_channels=out2_size, out_channels=out3_size,
kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(out3_size),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.AdaptiveAvgPool2d((10, 10))
        )

        self.classifier=nn.Sequential(
            nn.Flatten(),
            nn.Linear(out3_size*10*10,512),
            nn.ReLU(),
            nn.Dropout(0.3),  # 随机丢弃30%的神经元，防止过拟合
            nn.Linear(512, num_classes) # output
        )

    def forward(self,x):
        x=self.conv1(x)
        x=self.conv2(x)
        x=self.conv3(x)

        x=self.classifier(x)
        return x
```

2.2 模型训练

在模型训练过程中，使用了交叉熵损失函数 `nn.CrossEntropyLoss()`，能够较好地衡量离散类别的损失。

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = CNN(num_classes=len(train_dataset.classes)).to(device)
criterion = nn.CrossEntropyLoss()  # 交叉熵
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

train_losses = []
train_accuracies = []
test_accuracies = []


scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)

for epoch in range(epochs):
    model.train()
    train_loss=0
    # 训练阶段
    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
```

```
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        train_loss += loss.item() * inputs.size(0)

    epoch_loss = train_loss/len(train_dataset)
    train_losses.append(epoch_loss)

    # 计算训练集准确率
    model.eval()
    train_correct = 0
    train_total = 0
    with torch.no_grad():
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            _, predicted = torch.max(outputs.data, 1)
            train_total += labels.size(0)
            train_correct += (predicted == labels).sum().item()
    train_acc = train_correct / train_total
    train_accuracies.append(train_acc)
```

3.创新点

3.1 随机丢弃部分神经元，防止过拟合

在得到全连接层之前，通过`nn.Dropout(0.3)`随机丢弃部分神经元，可以防止过拟合

```
self.classifier=nn.Sequential(
            nn.Flatten(),

            nn.Linear(out3_size*10*10,512),
            nn.ReLU(),
            nn.Dropout(0.3),  # 随机丢弃30%的神经元，防止过拟合
            nn.Linear(512, num_classes) # output
        )
```

3.2 动态调整学习率

在训练初期，学习率要较大，而到了后期，学习率要逐渐减小，防止跳过最优解。下面用`torch.optim.lr_scheduler.StepLR`实现这一功能。

```
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)
```

通过动态调整学习率，可以使得模型最后收敛得较好，避免了最后波动较大的情况。

4.完整代码

```
import torch
```

```python
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np
import os

# 基础参数
epochs=30  # 迭代次数
batch_size = 16  # 批处理规模
learning_rate=0.001  # 学习率

out1_size=16
out2_size=32
out3_size=64

class CNN(nn.Module):
    def __init__(self,num_classes):
        super().__init__()
        self.conv1=nn.Sequential(

 nn.Conv2d(in_channels=3,out_channels=out1_size,kernel_size=3,stride=1,padding=1)
,
            nn.BatchNorm2d(out1_size),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(in_channels=out1_size, out_channels=out2_size,
kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(out2_size),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.conv3=nn.Sequential(
            nn.Conv2d(in_channels=out2_size, out_channels=out3_size,
kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(out3_size),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.AdaptiveAvgPool2d((10, 10))
        )

        self.classifier=nn.Sequential(
            nn.Flatten(),

            nn.Linear(out3_size*10*10,512),
            nn.ReLU(),
            nn.Dropout(0.3),  # 随机丢弃30%的神经元，防止过拟合
            nn.Linear(512, num_classes) # output
        )

    def forward(self,x):
        x=self.conv1(x)
        x=self.conv2(x)
```

```
        x=self.conv3(x)

        x=self.classifier(x)
        return x

def main():
    '''----------------------------------导入数据----------------------------------
_'''
    transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.RandomHorizontalFlip(),
        transforms.RandomRotation(10),
        transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
        transforms.ToTensor(),
        transforms.Normalize([0.5] * 3, [0.5] * 3)
    ])

    # 数据集路径
    base_path = r'D:\Python_code\作业\HW7\cnn图片'  # 我的路径
    train_path = os.path.join(base_path, 'train')  # 训练集
    test_path = os.path.join(base_path, 'test')  # 测试集

    # 加载数据集
    train_dataset = datasets.ImageFolder(train_path, transform=transform)
    test_dataset = datasets.ImageFolder(test_path, transform=transform)
    print(f"训练集类别: {train_dataset.class_to_idx}")
    print(f"测试集类别: {test_dataset.class_to_idx}")

    # 创建数据加载器
    train_loader = DataLoader(
        train_dataset,
        batch_size=batch_size,
        shuffle=True,
        num_workers=2
    )
    test_loader = DataLoader(
        test_dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers=0
    )

    '''--------------------------------初始化模型--------------------------------
'''
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    model = CNN(num_classes=len(train_dataset.classes)).to(device)
    criterion = nn.CrossEntropyLoss()  # 交叉熵
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

    train_losses = []
    train_accuracies = []
    test_accuracies = []


    scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5,
gamma=0.5)
```

```python
for epoch in range(epochs):
    model.train()
    train_loss=0
    # 训练阶段
    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        train_loss += loss.item() * inputs.size(0)

    epoch_loss = train_loss/len(train_dataset)
    train_losses.append(epoch_loss)

    # 计算训练集准确率
    model.eval()
    train_correct = 0
    train_total = 0
    with torch.no_grad():
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            _, predicted = torch.max(outputs.data, 1)
            train_total += labels.size(0)
            train_correct += (predicted == labels).sum().item()
    train_acc = train_correct / train_total
    train_accuracies.append(train_acc)

    # 计算测试集准确率
    test_correct = 0
    test_total = 0
    with torch.no_grad():
        for inputs, labels in test_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            _, predicted = torch.max(outputs.data, 1)
            test_total += labels.size(0)
            test_correct += (predicted == labels).sum().item()
    test_acc = test_correct / test_total
    test_accuracies.append(test_acc)

    print(f'Epoch [{epoch + 1}/{epochs}] Loss: {epoch_loss:.4f} '
          f'Train Acc: {train_acc:.4f} Test Acc: {test_acc:.4f}')
    scheduler.step()

# 绘制曲线
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(train_losses, label='Training Loss')
plt.title('Training Loss Curve')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
```

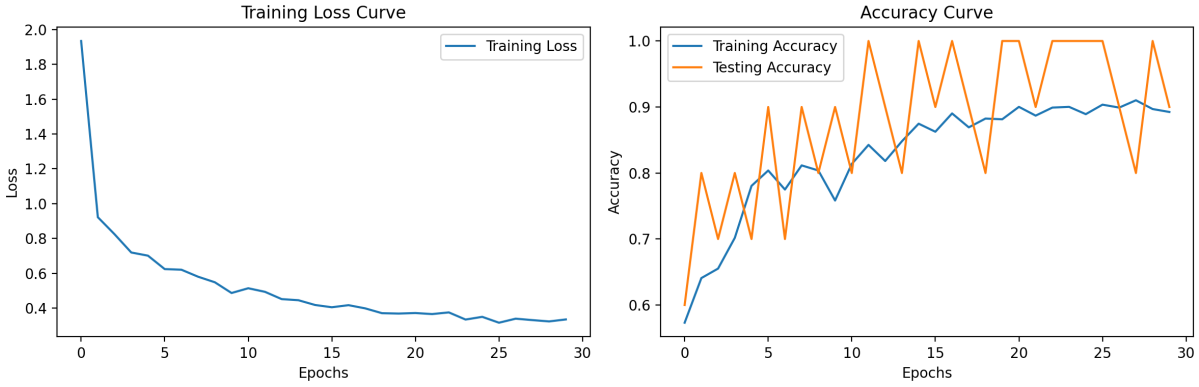
```
    plt.subplot(1, 2, 2)
    plt.plot(train_accuracies, label='Training Accuracy')
    plt.plot(test_accuracies, label='Testing Accuracy')
    plt.title('Accuracy Curve')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()

    plt.tight_layout()
    plt.show()

if __name__ == '__main__':
    torch.multiprocessing.freeze_support()

    # 设置较低的内存限制
    os.environ['OMP_NUM_THREADS'] = '1'
    # 我的电脑没有GPU
    os.environ['CUDA_VISIBLE_DEVICES'] = '-1'

    main()
```

三、实验结果及分析

设置迭代次数`epochs`为30，批处理规模`batch_size`为16，初始学习率`learning_rate`为0.001

```
# 基础参数
epochs=30  # 迭代次数
batch_size = 16  # 批处理规模
learning_rate=0.001  # 学习率
```

模型训练的损失函数图像，以及训练集和测试集的精确度如下所示：



由图可见，模型训练过程中的损失下降较快，最终低于40%。训练集的准确率逐渐升高，最后稳定在90%左右。最终，测试集的准确率达到了90%。（注意：这里，这里我没有把测试集用来训练了，而是每一轮训练后将测试集数据代入模型，计算测试集准确率，从而便于根据准确率的变化情况来选择合适的迭代次数）

根据这些统计数据可知，模型的训练效果比较好。但是由于测试集数量太少，总共只有10个，导致最终测试集的准确率具有一定的不稳定性。

四、参考文献

[1] 【Pytorch】2024 Pytorch基础入门教程（完整详细版）-CSDN博客

[2] PyTorch中基础模块torch的详细介绍_python torch-CSDN博客